

Språk = en (typisk uendelig) mengde av endelige strenger

Grammatikk = en **endelig** representasjon av et språk

$\langle \mathbf{N}(\text{onterminals}), \mathbf{T}(\text{erminals}), \mathbf{S} \in \mathbf{N}, \mathbf{R}(\text{ules}) \rangle$,

KontekstFri Grammatikk (CFG): $R \subset N \times (N + T)^*$

f.eks.: $\langle \{E, Int, Digit\}, \{0, 1, \dots, 9, +, *, (,)\}, E, R \rangle$

BNF :

EBNF :

$R = \{ E \rightarrow E + E$

$E \rightarrow E (+ \mid *) E \mid (E) \mid Int$

$E \rightarrow E * E$

$E \rightarrow (E)$

$E \rightarrow Int$

$Int \rightarrow Digit$

$Int \rightarrow Digit \{ Digit \}$

$Int \rightarrow Digit Int$

$Digit \rightarrow 0$

$Digit \rightarrow 0 \mid 1 \mid \dots \mid 9$

$Digit \rightarrow 1$

$\dots \rightarrow \dots$

$Digit \rightarrow 9 \}$

$\dots Int.Int \mid .Int \dots : [Int.]Int$

Entydighet

$$E \rightarrow E (+ | *) E | (E) | \text{Int}$$

1. derivasjoner (venstre/leftmost): $2 * 3 + 4$ vs. $(2 * 3) + 4$
2. parsetrær
3. tvetydighet – minst to parsetrær
 - (uavgjørbart for CFG)
4. abstrakte syntakstrær(abstrakt-syntaks)trær, ASTer
5. assosiativitet
 - venstre, f.eks., $-, +, *, /$: $1 - 2 - 3 = (1 - 2) - 3 \neq (1 - (2 - 3))$
 - eller høyre, f.eks.: eksp. $2^{3^2} = 2^{(3^2)} \neq (2^3)^2$
6. og presedens ...

Assosiativitet og presedens

høyre: $E \rightarrow Int \wedge E \mid Int$ vs. venstre: $E \rightarrow E - Int$

Hvert presedensnivå krever et nytt ikke-terminal symbol, samt et ekstra symbol for laveste elemeter:

$E \rightarrow E + E$	$E \rightarrow T + E \mid T$ svakest; høyre assos.
$.. \rightarrow ...$	$.. \rightarrow ...$
$E \rightarrow E * E$	$T \rightarrow G * T \mid G$ sterkest; høyre assos.
$E \rightarrow Int \mid (E)$	$G \rightarrow Int \mid (E)$

Deriver: $2 * 3 + 4$ og $2 + 3 * 4$

På hvert (presedens)nivå, må alle symbolene assosiere likt

$E \rightarrow G * E \mid E / T \mid G \mid T$, med $G, T \rightarrow ...Int$, gir tvetydighet:

$2 * 3 / 4$ er enten $2 * (3 / 4)$ eller $(2 * 3) / 4$.

Pre- og postfiks notasjon (polsk og omvendt polsk)

Blir $E \rightarrow + E E \mid * E E$ tvetydig ?

Vil $E \rightarrow E E * \mid E E +$ bli det ?

Prefiks og postfiks notasjon inneholder nøyaktig den samme informasjonen som ASTer:

- de er en linær representasjon av ASTer
- dermed blir parsetrær med en gang ASTer

1. Leksikalsk analyse
2. Parsing – bygg AST
3. bruk AST: evaluering,...

Leksikalsk analyse – tokenise :: String -> [String]

- ▶ “012 * (3 + 11)” må deles i meningsfulle **delstrenger** (tokens)

f.eks. [“012”, “*”, “(”, “3”, “+”, “11”, “)”]

- ▶ tokenise [] = []

tokenise (' ':xs) = tokenise xs

tokenise '('(:xs) = "(" : tokenise xs

tokenise ')'(:xs) = ")" : tokenise xs

tokenise (*':xs) = "*" : tokenise xs

...

tokenise (x:xs) = if isDigit x then

 (takeWhile isDigit (x:xs)) : tokenise (dropWhile isDigit xs)

 else **error** (“Ulovlig tegn ” ++ [x])

1. Leksikalsk analyse
2. Parsing – bygg AST

Parsing av språk definert ved prefiks gramatikk

$E \rightarrow + E E \mid * E E \mid \text{Int}$

data Ast = A Ast Ast | M Ast Ast | V Int

► parseE :: [String] -> Ast(Ast, [String])

```
parseE ("+":xs) = let e1 = parseE xs
                  e2 = parseE xs in A e1 e2 let (e1, r1)
= parseE xs ;
                  (e2, r2) = parseE r1 in (A e1 e2, r2)
```

```
parseE ("*":xs) = let (e1, r1) = parseE xs ;
                  (e2, r2) = parseE r1 in (M e1 e2, r2)
```

```
parseE (x:xs) = if (onlyDigits x) (V (read x :: Int), xs)
```

```
parseE (x:xs) = error ("Syntaksfeil ved " ++ x)
```

```
onlyDigits x = takeWhile isDigit x == x
```

- parse s = let (t,r) = parseE (tokenise s) in
if (null r) then t else error ("Feil ved " ++ (head r))
- evaluer, skriv ut i innfiks, ...

Recursive descent, predictive parsing

Grammatikk: startsymbolet E,

$$E \rightarrow E + E$$

$$E \rightarrow T + E \mid T$$

$$E \rightarrow E * E$$

$$T \rightarrow G * T \mid G$$

$$E \rightarrow - E$$

$$G \rightarrow - G \mid F$$

$$E \rightarrow \text{Int} \mid (E)$$

$$F \rightarrow \text{Int} \mid (E)$$

AST:

data Ast = F Int | G Ast | T Ast Ast | E Ast Ast
data Ast = **V** Int |
N Ast | **M** Ast Ast | **A** Ast Ast

parse :: [String] -> Ast

tokenise :: String -> [String]

så

men også...

`parse :: [String] -> (Ast, [String])`

$E \rightarrow T + E \mid T$ `parseE (s) → A Ast Ast , [String]`

$T \rightarrow G * T \mid G$ `parseT (s) → M Ast Ast , [String]`

$G \rightarrow - G \mid F$ `parseG (s) → N Ast , [String]`

$F \rightarrow \text{int} \mid (E)$ `parseF (s) → V Int , [String]`

`data Ast = V Int | N Ast | A Ast Ast | M Ast Ast`

parse :: [String] -> (Ast,[String])

- ▶ $\text{parseE}(s) = \text{let } (a,z) = \text{parseT}(s) \text{ in}$ $E \rightarrow T + E \mid T$
if null z then (a,z)
else if head(z) == "+" then
 let (c,rest) = $\text{parseE}(\text{tail}(z))$ in (A a c, rest)
else (a,z) -- hvis rekursjon
- ▶ $\text{parseT}(s) = \text{let } (a,z) = \text{parseG}(s) \text{ in}$ $T \rightarrow G * T \mid G$
if null z then (a,z)
else if head(z) == "*" then
 let (c,rest) = $\text{parseT}(\text{tail}(z))$ in (M a c, rest)
else (a,z) -- hvis rekursjon
- ▶ $\text{parseG} \text{ ("":s)} = \text{let } (a,b) = \text{parseG}(s) \text{ in } (N \ a, b)$ $G \rightarrow -G \mid F$
 $\text{parseG}(s) = \text{parseF}(s)$
- ▶ $\text{parseF} \text{ ("":s)} = \text{let } (a, \text{"":b}) = \text{parseE}(s) \text{ in } (a, b)$ $F \rightarrow \text{Int} \mid (E)$
 $\text{parseF}(x:s) = (V(\text{read } x), s)$ -- $(\text{read } x) :: \text{Int}$ kan trenge
- ▶ $\text{parse } s = \text{let } (t, r) = \text{parseE}(\text{tokenise } s) \text{ in}$
if (null r) then t else error ("Feil ved " ++ (head r))

1. Leksikalsk analyse
2. Parsing – bygger AST
3. bruk AST: evaluering,...

$\text{data Ast} = \text{V Int} \mid \text{N Ast} \mid \text{A Ast Ast} \mid \text{M Ast Ast}$

$\text{eval} :: \text{Ast} \rightarrow \text{Int}$

$\text{eval (V } x) = x$

$\text{eval (A } v \text{ h)} = (\text{eval } v) + (\text{eval } h)$

$\text{eval (N } t) = -(\text{eval } t)$

$\text{eval (M } v \text{ h)} = (\text{eval } v) * (\text{eval } h)$

$\text{skriv} :: \text{Ast} \rightarrow \text{String}$

$\text{skriv (V } x) = \text{show } x$ $\text{skriv (A } v \text{ h)} = '(:(\text{skriv } v)++'+':(\text{skriv } h) ++")'$

$\text{skriv (N } t) = '-' : \text{skriv } t$ $\text{skriv (M } v \text{ h)} = '(:(\text{skriv } v)++'*':(\text{skriv } h) ++")'$

$\text{foldt} :: (\text{Int} \rightarrow e) \rightarrow (e \rightarrow e) \rightarrow (e \rightarrow e \rightarrow e) \rightarrow (e \rightarrow e \rightarrow e) \rightarrow \text{Ast} \rightarrow e$

$\text{foldt } v \text{ n a m (V } x) = v \text{ } x$

$\text{foldt } v \text{ n a m (N } t) = \text{n } ? (\text{foldt } v \text{ n a m } t)$

$\text{foldt } v \text{ n a m (A } l \text{ r)} = \text{a } (\text{foldt } v \text{ n a m } l) (\text{foldt } v \text{ n a m } r)$

$\text{foldt } v \text{ n a m (M } l \text{ r)} = \text{m } (\text{foldt } v \text{ n a m } l) (\text{foldt } v \text{ n a m } r)$

$\text{ev} = \text{foldt id (0-)} (+) (*)$

$\text{sk} = \text{foldt show ('-')} (\backslash \text{vr hr} \rightarrow '(: \text{vr} ++ '+' : \text{hr} ++ ")')$
 $\quad (\backslash \text{vr hr} \rightarrow '(: \text{vr} ++ '*' : \text{hr} ++ ")')$

$\text{pre} = \dots$

Lett – LL(k)

LL grammatikk – tillatter en parser som leser input fra venstre (Left) og gradvis konstruerer venstre-derivasjon (Leftmost)

- ▶ recursive descent, predictive parsing

$\text{parseE}(s) = \text{let } (a,z) = \text{parseT}(s) \text{ in} \quad E \rightarrow T + E \mid T$
if null z then (a,z)
else if head(z) == "+" then
let (c,rest) = $\text{parseE}(\text{tail}(z))$ in (A a c, rest) else...

- ▶ ingen venstre-rekursjon! $E ::= E + P \dots$

$\text{parseE}(s) = \text{let } (a,z) = \text{parseE}(s) \text{ in} \dots$

LL(1) 1-lookahead; nok med å lese neste symbol/token, for å bestemme neste derivasjonsteg, f.eks.:

$E ::= T * E \mid T / E \mid T$

$E ::= P + E \mid P + E + aF \mid P + E + bT \dots$

LL(k) nok med å lese k tokens/symboler foran

- ▶ parsing av LL(k) krever ikke backtracking

Monadisk parsing

- bokens kappitel 12
- en forelesning 'Functional Parsing' av Graham Hutton på Computerphile:

<https://www.youtube.com/watch?v=dDtZLm7HIJs>

- en god forklaring av generelle trekk ved funksjonell parsing,
- med elementer av monadisk parsing;
ikke en fullstendig introduksjon

<https://www.youtube.com/watch?v=SToUyjAsaFk>

- en enkel innføring i rekursiv descent parsing

kommer om et par uker ...

Det gjenstår presentasjon av typeavledning, nemlig,

- (1) unifikasjonsalgoritme, som danner basis for
- (2) Hindley-Milner algoritme